



1 VISTAS EN SQL

Las vistas (sintaxis) son tablas virtuales basadas en el resultado de una consulta.

No almacenan datos, solo presentan los datos de una o varias tablas.

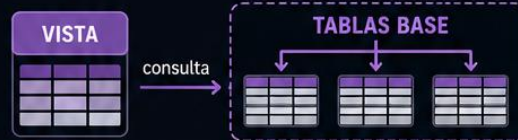
```
SELECT *  
FROM vista;  
-- Consulta  
simplificada
```

¿QUÉ ES UNA VISTA?

Una vista es una tabla virtual basada en el resultado de una consulta.

- ✓ No almacena datos propios. Refleja los datos actuales de las tablas base.

¿CÓMO FUNCIONA?



- 💡 La vista ejecuta la consulta cada vez que se usa, mostrando siempre la información más actual.

SINTAXIS BÁSICA

```
CREATE VIEW nombre_vista AS  
SELECT columna1, columna2, ...  
FROM tabla  
WHERE condición;  
  
-- La vista se consulta como una tabla normal  
SELECT * FROM nombre_vista;
```

VENTAJAS

- SEGURIDAD**
Permite limitar el acceso a ciertos datos y ocultar columnas sensibles.
- SIMPLICIDAD**
Simplifica consultas complejas y reutiliza lógica de negocio.
- CONSISTENCIA**
Siempre refleja la información actual de las tablas base.

EJEMPLO PRÁCTICO

Supongamos que tenemos una tabla Empleados y queremos mostrar solo el nombre y el salario.

1. CREAR LA VISTA

```
CREATE VIEW Vista_Empleados AS  
SELECT Nombre, Salario  
FROM Empleados;
```

- 💡 La vista muestra los datos como si fuera una tabla, pero no almacena información, solo consulta.

2. USAR LA VISTA

```
SELECT *  
FROM Vista_Empleados;
```

Nombre	Salario
Ana Pérez	4500.00
Luis Gómez	5200.00
María Ruiz	4800.00

¿DÓNDE SE USAN?

- Reportes y análisis de datos
- Restricción de acceso a datos sensibles
- Simplificación de consultas complejas
- Adaptación a cambios sin modificar aplicaciones

BUENAS PRÁCTICAS

- ✓ Usa nombres descriptivos para las vistas.
- ✓ No incluyas lógica innecesariamente compleja.
- ✓ Documenta el propósito de cada vista.
- ✓ Revisa el rendimiento en consultas grandes.



```
-- Las vistas son una herramienta poderosa para organizar, proteger y simplificar  
-- el acceso a los datos en SQL.
```

SQL >_

2

FUNCIONES (SINTAXIS)

Las funciones son bloques de código que realizan una tarea específica y pueden devolver un valor.



TIPOS DE FUNCIONES



FUNCIÓN ESCALAR

Devuelve un solo valor.

```
CREATE FUNCTION nombre_funcion
(@parametro tipo_dato)
RETURNS tipo_dato
AS
BEGIN
    -- instrucciones
    RETURN valor
END;
```

EJEMPLO

Función que devuelve el IVA (18%) de un valor.

```
CREATE FUNCTION fn_IVA
(@monto DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @monto * 0.18
END;
```

USO: `SELECT dbo.fn_IVA(1000) AS IVA;`

RESULTADO:

IVA
180.00



FUNCIÓN DE TABLA

Devuelve un conjunto de filas (tabla).

```
CREATE FUNCTION nombre_funcion()
RETURNS TABLE
AS
RETURN
(
    -- consulta que devuelve
    -- un conjunto de filas
);
```

EJEMPLO

Función que devuelve empleados con salario mayor a 3000.

```
CREATE FUNCTION fn_EmpleadosAltos()
RETURNS TABLE
AS
RETURN
(
    SELECT IdEmpleado, Nombre, Salario
    FROM Empleados
    WHERE Salario > 3000
);
```

USO: `SELECT * FROM dbo.fn_EmpleadosAltos();`

RESULTADO:

IdEmpleado	Nombre	Salario
2	Luis Gómez	4500.00
3	Ana Pérez	5200.00



USOS COMUNES

- ✓ Reutilización de código.
- ✓ Encapsular lógica de negocio.
- ✓ Mejorar la legibilidad y mantenimiento.
- ✓ Facilitar cálculos y transformaciones.
- ✓ Reducir duplicación de consultas complejas.



RECUERDA: Las funciones no modifican datos directamente. Solo devuelven un valor (escalar) o un conjunto de filas (tabla) que puedes usar en consultas.



3

ESQUEMAS

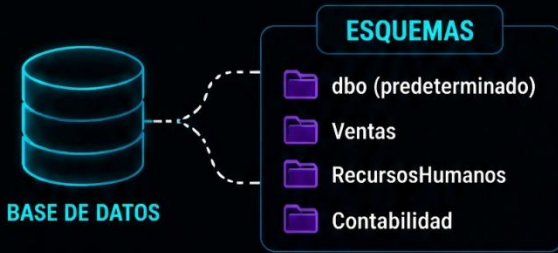


Un esquema es un **contenedor** de objetos de base de datos (tablas, vistas, funciones, etc.). Permite organizar y administrar los objetos de forma lógica.



¿CÓMO FUNCIONA?

Cada usuario puede tener su propio esquema para almacenar y administrar sus objetos, separándolos de otros usuarios.



CARACTERÍSTICAS

- ✓ **Organización:** permite agrupar objetos relacionados.
- ✓ **Seguridad:** facilita la asignación de permisos a nivel de esquema.
- ✓ **Administración:** simplifica el mantenimiento y control de objetos.
- ✓ **Evita conflictos:** permite usar nombres iguales de objetos en diferentes esquemas.



EJEMPLO

Supongamos que tenemos una base de datos llamada **Empresa** y queremos crear un esquema llamado **Ventas** para almacenar las tablas relacionados.

1 CREAR UN ESQUEMA

```
CREATE SCHEMA Ventas;
```

- ✓ **Resultado:** El esquema Ventas se crea dentro de la base de datos Empresa.

Base de datos: Empresa

Esquema: dbo
(predeterminado)



Esquema: Ventas
(nuevo)



2 USAR EL ESQUEMA

Creamos una tabla dentro del esquema Ventas.

```
CREATE TABLE Ventas.Clientes (  
  IdCliente INT PRIMARY KEY,  
  Nombre VARCHAR(100),  
  Ciudad VARCHAR(100)  
);
```

- ✓ **Resultado:** La tabla Clientes pertenece al esquema Ventas.

Base de datos: Empresa

Esquema: dbo



Esquema: Ventas



Clientes



BENEFICIOS



Mejora la seguridad permite controlar accesos por esquema.



Favorece la organización de los objetos de la base de datos.



Facilita el trabajo en equipos, separando funciones y objetos.



Evita conflictos de nombres entre usuarios.



RECUERDA: El esquema dbo es el predeterminado en SQL Server y pertenece al propietario de la base de datos (generalmente el usuario sa).



4

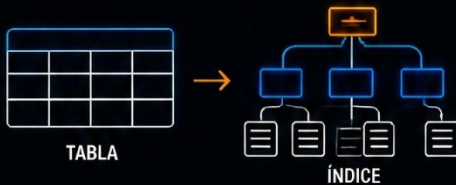
ÍNDICES

Un índice es una estructura que mejora la velocidad de búsqueda y recuperación de datos en una tabla.



¿QUÉ ES UN ÍNDICE?

Es una estructura auxiliar que almacena los valores de una o más columnas en un orden específico junto con las referencias a las filas de la tabla. Permite encontrar los datos más rápido sin tener que recorrer toda la tabla (búsqueda completa).



SINTAXIS BÁSICA

```
CREATE INDEX nombre_indice
ON tabla (columna1, columna2)
[UNIQUE] [CLUSTERED | NONCLUSTERED];
```

- UNIQUE:** garantiza que los valores de la columna o columnas sean únicos.
- CLUSTERED:** el índice define el orden físico de los datos en la tabla (solo puede haber uno por tabla).
- NONCLUSTERED:** el índice es una estructura separada que contiene punteros a los datos.

TIPOS DE ÍNDICES

CLUSTERED INDEX

Ordena físicamente los datos de la tabla. Solo puede existir uno por tabla.



Ideal para columnas con valores únicos y consultadas frecuentemente.

NONCLUSTERED INDEX

Estructura separada que contiene punteros a las filas de la tabla.



Puede haber varios por tabla.

UNIQUE INDEX

Asegura que los valores en la columna(s) sean únicos.



Puede ser CLUSTERED o NONCLUSTERED.

BENEFICIOS

- Mejora la velocidad de búsqueda y consultas.
- Optimiza el rendimiento de operaciones JOIN, WHERE y ORDER BY.
- Reduce el tiempo de respuesta en aplicaciones.



DESVENTAJAS

- Ocupa espacio adicional en disco.
- Puede hacer más lentas las operaciones de INSERT, UPDATE y DELETE.
- Requiere mantenimiento y actualización.



EJEMPLO PRÁCTICO

Supongamos que tenemos una tabla Empleados y queremos crear diferentes índices.

Tabla: Empleados

idEmpleado	Nombre	Departamento	Salario
1	Ana Pérez	Ventas	3500.00
2	Luis Gómez	Sistemas	5200.00
3	María Ruiz	Ventas	4800.00
4	Juan Soto	Sistemas	4100.00

- 1 Crear índice NONCLUSTERED en la columna Departamento

```
CREATE INDEX idx_Departamento
ON Empleados (Departamento);
```

Uso:
SELECT * FROM Empleados
WHERE Departamento = 'Ventas';

- 2 Crear índice UNIQUE en la columna idEmpleado

```
CREATE UNIQUE INDEX idx_idEmpleado
ON Empleados (idEmpleado);
```

Uso:
SELECT * FROM Empleados
WHERE idEmpleado = 2;

- 3 Crear índice CLUSTERED en la columna Salario

```
CREATE CLUSTERED INDEX idx_Salario
ON Empleados (Salario);
```

Uso:
SELECT * FROM Empleados
ORDER BY Salario;

MEJORES PRÁCTICAS



Crea índices en columnas que se usan en búsquedas frecuentes.



Evita crear índices innecesarios.



Revisa y elimina índices no utilizados.



Monitorea el rendimiento y ajusta según sea necesario.



RECUERDA:

Los índices mejoran la velocidad de lectura, pero pueden afectar la velocidad de escritura y ocupar espacio adicional. Usalos de forma estratégica.

